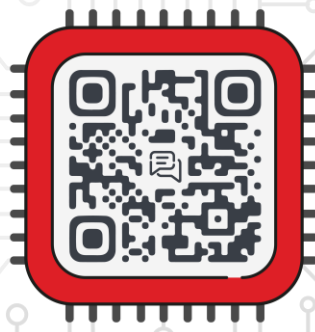
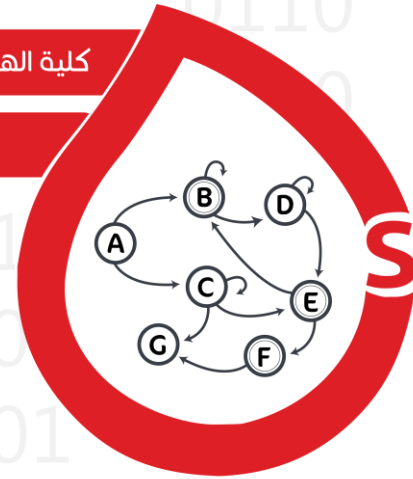




Turing Machine

Part 1

د. هبة حاتم



اللغات الصورية

تذكرة:

في هرمية chomsky درسنا القواعد:

- type 3 وهي ال regular grammer و التي تدركها ال FSM

- type 2 وهي ال context free grammar و تدركها ال PDA

وسندرس في آخر محاضرتين من المقرر ال Unristicted grammer و التي تدركها ال Turing machine

أما بالنسبة لقواعد النوع 1 لا يتم تدريسها في المقررات لأنها معقدة جدا و ال turing machine أبسط منها

- القواعد من النوع 2 يمكنها أيضا أن تدرك اللغات من النوع 3 ولكن العكس غير صحيح

Unrestricted grammar

- هي القواعد من النوع 0 و هي الأعم حسب هرمية Chomsky
- قواعد الإنتاج فيها تكون بالكامل غير مقيدة طالما أن الجزء الأيسر منها ليس فارغاً
- حيث $\alpha \rightarrow B, \alpha \in (V \cup \Sigma)^*$ و $\varepsilon \neq \alpha$
- تدرك هذه القواعد اللغات القابلة للتعداد بشكل عودي (RE Recursively Enumerable Languages)
- وتسمى أيضاً Turing-recognizable languages

Computability vs. Decidability

Computability (can we compute a function) ❖

وتعني ما إذا كانت مشكلة قابلة للحل/للحساب باستخدام خوارزمية معينة (مثل TM) تكون الدالة قابل للحساب إذا وجدت TM تعطي خرج صحيح دائماً لأي دخل صالح (valid) قد تحتاج بعض الوقت لكن في النهاية ستتوقف و تعطي الجواب الصحيح



مثال 1 :

الدالة $f(n) = n^2$ قابلة للحساب، يمكننا كتابة برنامج لحلها

مثال 2:

الدالة $f(n)$ تكون 1 إذا كانت فرضية غولدمباك صحيحة، وإلا 0.
هذه الدالة غير معروفة بأنها قابلة للحساب، لأننا لا نعرف ما إذا كان بإمكاننا كتابة خوارزمية تتوقف دائماً وتعطينا الإجابة.

فرضية غولدمباخ هي واحدة من أشهر المسائل غير المحلولة في الرياضيات، وتنص على:
كل عدد زوجي أكبر من 2 يمكن كتابته على شكل مجموع عددين أوليين".

أمثلة:

$$4 = 2 + 2$$

$$6 = 3 + 3$$

$$8 = 3 + 5$$

$$10 = 5 + 5 \text{ أو } 7 + 3$$

التحقق من فرضية غولدمباخ:

- إذا كانت صحيحة، يمكن أن نحاول التحقق لكل عدد زوجي، لكن هناك عدد لانهائي من الأعداد الزوجية.
- إذا كانت خاطئة، لا نعرف العدد الزوجي الذي يكسرها، لذلك لن نتوقف الخوارزمية أبداً حتى تجده.

❖ Decidability (can we answer a yes/no question with certainty?)

- القابلية للقرار (Decidability) هي حالة خاصة من القابلية للحساب — وهي تتعلق بما إذا كان من الممكن الإجابة على مسألة قرار (سؤال بنعم/لا) بطريقة خوارزمية
- المسألة تكون قابلة للقرار إذا كانت آلة تورنغ دائماً تتوقف وتعطي إجابة بنعم أو لا.
- إذا لم توجد خوارزمية تتوقف دائماً وتعطي الإجابة الصحيحة بنعم أو لا، فإن المسألة غير قابلة للقرار

مثال 1:

"هل العدد المعطى أولي؟" هناك خوارزميات للإجابة على هذا السؤال، إذن المسألة قابلة للقرار.

مثال 2 :

"هل سيتوقف هذا البرنامج العشوائي عند هذا المدخل؟" (مسألة التوقف)
هذه المسألة غير قابلة للقرار — لا توجد خوارزمية يمكنها الإجابة على جميع الحالات.

Turing Machines

هي آلات مجردة - مفاهيمية و ليست فيزيائية - و يمكنها محاكاة أي حاسوب في وقتنا الحالي لكن ببطء شديد مقارنة بهم

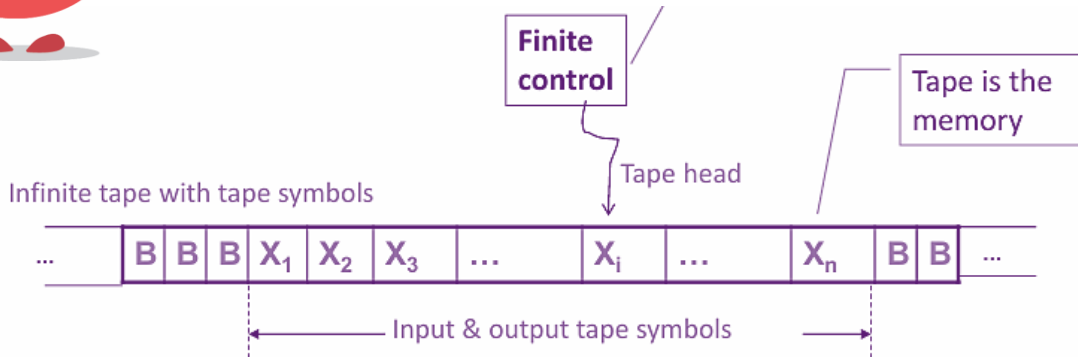
ما فائدة هذه الآلات؟

إذا كانت المشكلة التي نحاول حلها لا يمكن حلها باستخدام ال TM (من أجل كل دخل يكون الجواب Yes or No) ، هذا يعني أن المشكلة undecidable

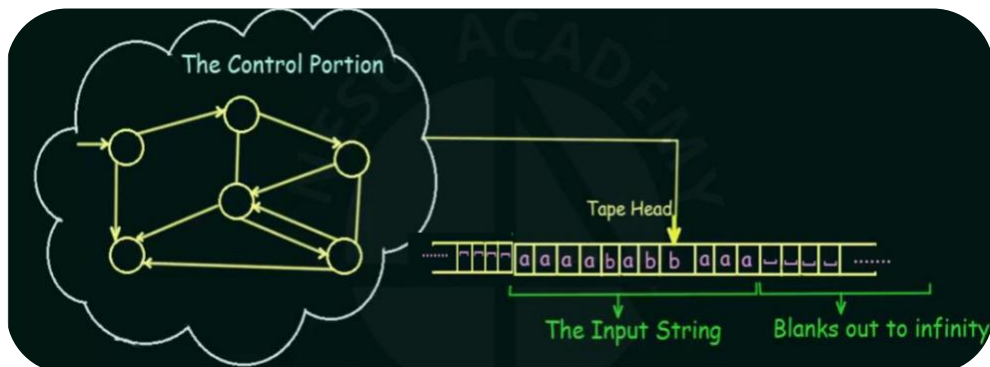
- آلة تورنغ هي FA مزود بشريط لانهائي يستخدم كذاكرة.
- يُكتب الدخل على الشريط عند بداية الحساب، ويكون محاطاً بعدد لانهائي من الخلايا الفارغة.
- كل انتقال يعتمد على الرمز الحالي الموجود تحت رأس الشريط.

تتكون آلة تورنغ من ثلاثة أجزاء:

1. وحدة تحكم ذات حالات محدودة finite-state control تصدر الأوامر،
2. شريط لانهائي للإدخال
3. رأس شريط يمكنه قراءة وكتابة خلية واحدة من الشريط .



الرمز B يشير إلى الفراغ blank وهو رمز محجوز لإعطاء حدود للبيانات
ممكن أن يتم ترميزها أيضاً ب □ أو ٠



- وحدة التحكم هنا مشابهة ل FSM (مشابهة لها و ليست FSM فعليا) وهي deterministic (يمكن بناء وحدة التحكم لتكون deterministic او non-deterministic لكن استخدام ال non-deterministic أقل بكثير و تعقيده أكبر)
- الرمز في الشريط من جهة اليمين يدل على وجود فراغ لا نهائي من هذه الجهة أما الرمز يدل على وجود حد هنا (غير مفتوح من جهة اليسار)

Formal Definition

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

حيث:

- Q : مجموعة منتهية من الحالات.
- Σ : أبجدية اللغة.
- Γ : أبجدية الشريط (والتي تحوي Σ)
- δ : تابع الانتقال.
- q_0 : الحالة الابتدائية.
- B : رمز الفراغ (blank symbol) وهو special symbol لا ينتمي لأبجدية اللغة
- F : مجموعة من الحالات النهائية (حالة مقبولة و حالة الرفض)



- يمكن وجود أكثر من accept state لكنها ليست الحالة الغالبة حيث عدم تعدد الحالات المقبولة قد يكون أسهل للتصميم.
- ال reject state ممكن أن تكون موجودة ظاهرياً أو ضمناً حيث عند وجود خطأ يتم الذهاب لحالة الرفض مباشرة.
- يتم التوقف عند الوصول ل accept state او reject state و عند الوصول ل accept state لا يتم الانتقال منها لأي حالة أخرى عكس ال DFA حيث توجد انتقالات من ال accept state فيها

تابع الانتقال

- لكل حركة انتقال في ال TM غالبا يتم ترميزها ب --- | يحدث التالي: $\delta(q, X) = (p, Y, D)$ حيث:
- q : الحالة الحالية.
 - X : رمز الشريط الحالي الذي يشير إليه الرأس.
 - الحالة تتغير من q إلى p
 - بعد الانتقال:

1- يتم استبدال الرمز X ب Y

- 2- إذا كان $D = "L"$ ينتقل الرأس إلى اليسار خطوة واحدة أما إذا كان $D = "R"$ فينتقل لليمين خطوة واحدة.
(يمكن أيضا التعبير عن L ب $\leftarrow$ ويمكن التعبير عن R ب $\rightarrow$)

كما يمكن التعبير عن تابع الانتقال بالتالي: $Q \times \Sigma \rightarrow \Gamma \times (R/L) \times Q$

و تعني عند حالة معينة و دخل معين يتم كتابة رمز من Γ على الشريط و الانتقال خطوة لليمين أو اليسار و الانتقال لحالة جديدة من Q (يمكن أن يبقى في نفس الحالة أيضاً)

ما يتم كتابته على الانتقالات في المخطط :

$X \rightarrow Y, D$ أو $X / Y, D$

أي عند قراءة X استبدله ب Y و تحرك للجهة D , إذا كنا لا نريد تغيير القيمة الحالية إذاً نعيد كتابة نفس الرمز مرة أخرى, مثال

$1 \rightarrow 1, L$ عند قراءة 1 قم باستبداله ب 1 و

تحرك خطوة لليسار

Rules of Operations

لكل خطوة من الحساب نقوم بالتالي:

- قراءة رمز في الخلية التي تحت رأس الشريط،
- الكتابة لتغيير الخلية الحالية
- تحريك الرأس خلية واحدة فقط لليمين او اليسار (وهذا وفقاً لوحدة التحكم)

آلية التحكم:

- تتحكم بها آلة شبيهة لل FSM
- تبدأ من حالة ابتدائية محددة مسبقاً.

حالات الإنهاء

هناك حالتان نهائيتان مميزتان:

- حالة القبول (ACCEPT STATE)
- حالة الرفض (REJECT STATE)

نواتج الحساب

يمكن للآلة أن تنتهي بإحدى ثلاث طرق:

- التوقف والقبول $\rightarrow$ (HALT and ACCEPT) إنهاء ناجح.
- التوقف والرفض $\rightarrow$ (HALT and REJECT) إنهاء غير ناجح.
- حلقة $\rightarrow$ (LOOP) الآلة لا تتوقف أبداً (حساب غير منتهٍ) .

أطروحة تورينغ Turing Thesis

أي عملية حسابية يمكن تنفيذها بوسائل ميكانيكية يمكن تنفيذها بواسطة آلة تورنغ".



أهم الحجج التي تدعم الأطروحة:

- محاكاة الحواسيب الرقمية:

أي شيء يمكن تنفيذه على الحواسيب الرقمية الحالية يمكن أيضاً تنفيذه بواسطة آلة تورنغ.

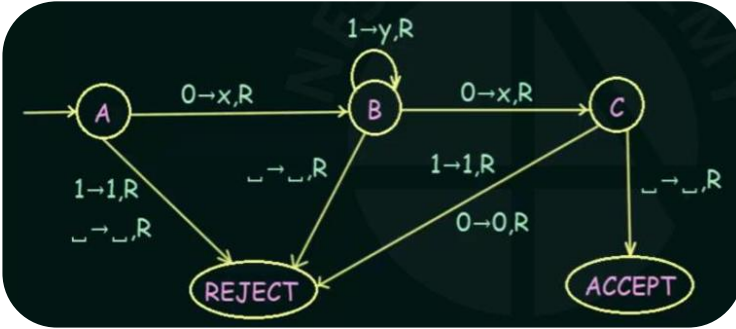
- لم يتم العثور حتى الآن على أي مسألة خوارزمية تعتبر قابلة للحل ولا توجد لها آلة تورنغ قادرة على حلها.

مثال 1:

صمم آلة تورينغ تدرك اللغة : $L = 01^*0$
(هذه اللغة Regular لكن ذكرنا سابقاً أن ال TM تدرك جميع اللغات)

الحل:

نتخيل الدخل كما رأينا سابقاً هو شريط و من اليمين و اليسار يوجد رمز الفراغ و في المنتصف تكون سلسلة الدخل التي سنختبرها و القراءة تتم من اليسار إلى اليمين و تبدأ من أول محرف من السلسلة وبما أن ال TM التي ندرسها deterministic بالتالي يجب وجود انتقال لكل رمز من الأبجدية لكل حالة عدا الحالات النهائية وعند عدم وضع انتقال لرمز معين لحالة معينة فإنه ينتقل لل reject state بشكل تلقائي



نلاحظ أن اللغة تمثل جميع السلاسل التي تبدأ وتنتهي ب 0 وبينهما عدد غير محدد من الواحدات و طول السلسلة 2 على الأقل (00)

- بالتالي عند الحالة الابتدائية إذا قرأنا 1 فإن هذا يؤدي للرفض و إذا قرأنا "فراغ" في البداية فهذا يعني أن السلسلة المدخلة فارغة و هذا غير مقبول بالتالي تؤدي للرفض أيضاً

- عند قراءة أول صفر نبدل ب X و ننتقل للحالة التالية

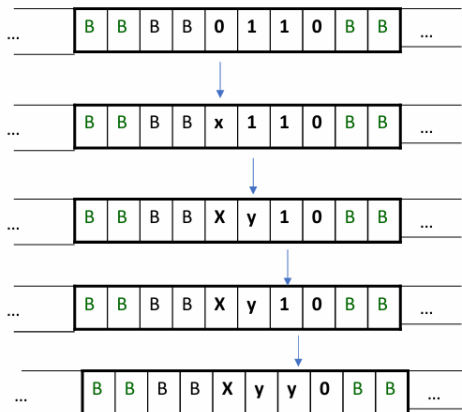
- بعدها نبقى في نفس الحالة ما دمنا نقرأ 1 حيث في كل مرة نستبدله ب Y و ننتقل خطوة لليمين

- أما إذا قرأنا بعد أول صفر "فراغ" مباشرة أو إذا قرأنا بعد عدد من الواحدات "فراغ" مباشرة أيضاً هذا يعني أن السلسلة لم تنتهي بصفر و يؤدي للرفض

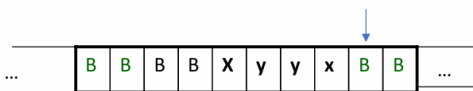
- أما إذا قرأنا صفر بعد عدد من الواحدات أو بعد الصفر الأول مباشرة نبدل ب X و يتحرك رأس الشريط لليمين خطوة و ننتقل لحالة أخرى لتختبر بعدها ما إذا كان هذا الصفر هو الأخير أم لا

- الحالة التي وصلنا لها الآن (عند C) تكون فيها السلسلة إما 01^*0 أو 00 بالتالي سواء قرأنا بعدها 0 أو 1 فإن شرط السلسلة سيختل و يؤدي للرفض

- أما إذا قرأنا "فراغ" فهذا يعني أننا وصلنا لنهاية السلسلة و هي مقبولة



• مخطط يوضح مراحل الحل على مثال 0110 :



ملاحظة

استبدال ال 0 ب x و ال 1 ب y ليس ضروري لهذا المثال, أي كان بإمكاننا أن نعوض بأي رمزين آخرين أو حتى أن نبدل بنفس الرمزين (أن يبقى الصفر صفر و الواحد واحد) لكن قمنا بالاستبدال برموز مغايرة كي يسهل علينا تتبع مراحل الحل

مثال 2:

أوجد TM للغة : $\{0^n 1^n \mid n \geq 1\}$

الحل:

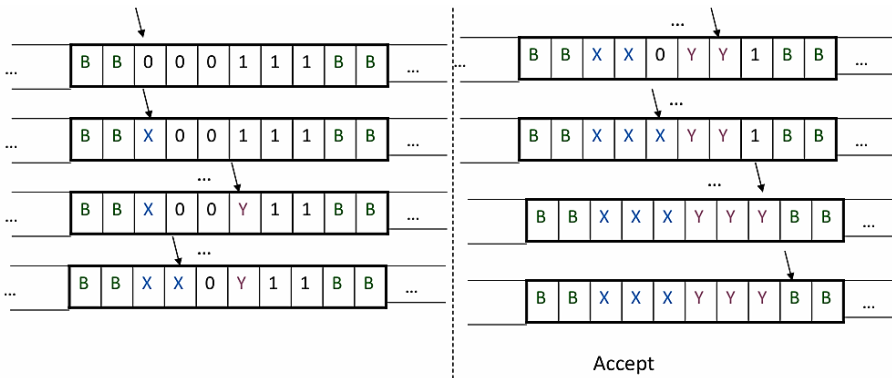
الفكرة الأساسية للحل هنا هو مطابقة كل 0 مع 1

1. نقوم باستبدال الصفر ب X و نتحرك خطوة لليمين
2. نستمر بالتحرك لليمين حتى نصادف 1 فنستبدله ب Y و نتحرك ليسار
3. نستمر بالتحرك ليسار حتى نصادف X عندها نتحرك خطوة لليمين (لا نريد تغيير قيمة X لذلك نقون باستبدالها بنفسها)
4. بعد التحرك لليمين: إذا كان العنصر 0 نعيد من الخطوة 1 وإلا نتحرك لليمين لتأكد من عدم وجود أي واحدات إذا لم نجد أي واحدات أي أننا وصلنا للنهاية عند عنصر فارغ فتتوقف الخوارزمية بنجاح

(نلاحظ عدم وجود reject state لكنها موجودة بشكل ضمني)

(لو كان السؤال $n \geq 0$ نضيف انتقال عند الرمز الفارغ للحالة المقبولة لنقبل السلسلة الفارغة)

- تتبع الحل على مثال: 000111



19

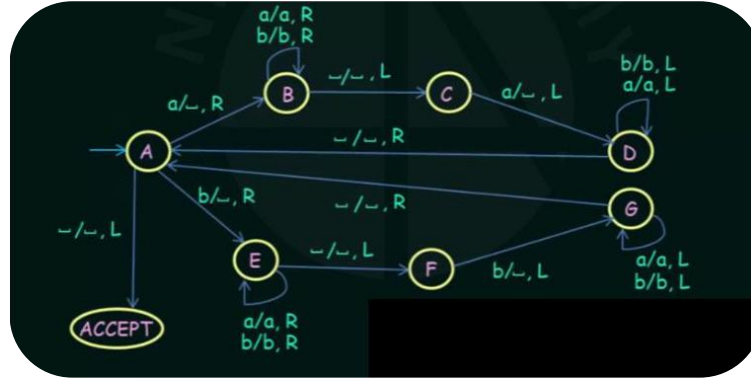
	Next Tape Symbol				
Curr. State	0	1	X	Y	B
q_0	(q_1, X, R)	-	-	(q_3, Y, R)	-
q_1	$(q_1, 0, R)$	(q_2, Y, L)	-	(q_1, Y, R)	-
q_2	$(q_2, 0, L)$	-	(q_0, X, R)	(q_2, Y, L)	-
q_3	-	-	-	(q_3, Y, R)	(q_4, B, R)
$*q_4$	-	-	-	-	-

تمثيل ال state diagram بجدول انتقالات:

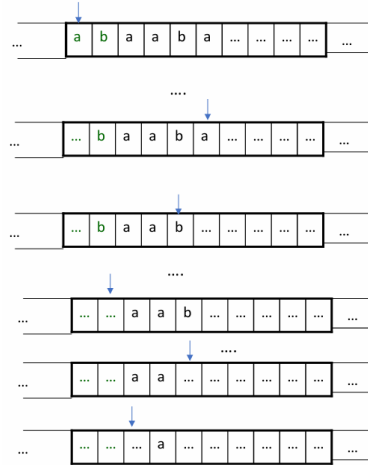
صمم TM تقبل لغة ال even palindromes باستخدام الأبجدية: $\Sigma = \{a, b\}$

الحل:

الفكرة الأساسية للحل: نطابق المحرف الأول مع الأخير ثم نطابق المحرف الثاني مع المحرف قبل الأخير و هكذا.. عند المطابقة نقوم بتعويض المحارف التي تم مطابقتها بالرمز الفارغ)



تتبع الحل على مثال: abaaba



عدة أشكال لآلة تورنغ

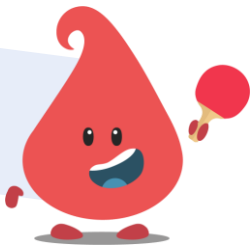
- شريط واحد أو عدة أشرطة
- لانهائي من الطرفين (يمكن أن يكون لانهائياً من الطرف الأيمن فقط)
- الأبجدية ليست بالضرورة 0 و 1
- يمكن للرأس أن يبقى في نفس المكان
- تسمى هذه "آلات تورنغ الثابتة" أو آلات تورنغ ثلاثية الاتجاه، حيث $DE \in \{L, R, S\}$ و S تعني التوقف (البقاء في نفس المكان) وهذا النوع قد يزيد التعقيد
- السماح بعدم الحتمية (Non-Determinism)

جميع الأشكال متكافئة في القدرة الحسابية:

إن توسيع آلة تورنغ الأساسية (مثل الأشرطة المتعددة، أو عدم الحتمية، أو الأبجديات الأكبر، إلخ) لا يزيد من قدرتها الحسابية. جميع الأشكال يمكنها حل نفس فئة المسائل بالضبط (تكافؤ القابلية للحساب) عندما نقول أن أمراً ما يزيد تعقيد الحل هذا لا يعني أنه لم يعد قادراً على حلها)

تقنيات برمجة آلة تورنغ

- عندما نتحدث عن تقنيات برمجة آلة تورنغ، فإننا نشير إلى الاستراتيجيات المستخدمة في بناء آلات تورنغ لتنفيذ عمليات حسابية مفيدة.
- فكر في الأمر كأنه "برمجة منخفضة المستوى" — بدلاً من كتابة الكود بلغة مثل Python أو JavaScript ، أنت تقوم بتصميم الانتقالات بين الحالات والتعامل مع الرموز على الشريط.
- **هناك أنماط أو حيل شائعة (سنتحدث عنها في المحاضرات القادمة) تُستخدم لجعل آلة تورنغ تنفذ مهام مثل:**



- نسخ الإدخال
- التحقق من الأنماط
- مقارنة القيم
- التكرار (Looping)
- وضع علامات على مواضع في الشريط
- التحكم في تدفق التنفيذ عبر انتقالات الحالات

Turing machines Programming Techniques

بعد أن درسنا آلات تيورنغ، سنتحدث الآن كيفية تعديل هذه الآلات لتناسب حالات معينة

مثال 1:

المشكلة المطروحة: كيف يمكننا تحديد النهاية اليسرى لشريط ال TM
لا يوجد شيء built in يحل هذه المشكلة لذلك سننفذها بأنفسنا

الحل:

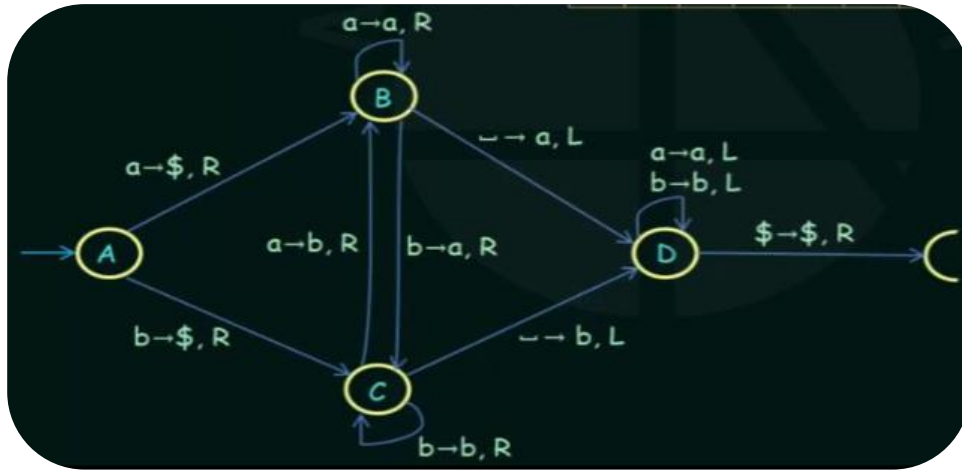
سنضع رمز \$ في النهاية اليسرى للشريط و نقوم بإزاحة كل العناصر لليمين خطوة:



والآن كيف سيكون تصميم TM للقيام بهذه العملية؟

- من الحالة A نأخذ بعين الاعتبار أن تكون بداية a أو b و بما أنها البداية فنستبدلها ب \$ و ننتقل لليمين
- إذا كان الرمز الأول a سنستبدل ب \$ و ننتقل لليمين و الآن إذا صادفنا a فنستبدله ب a (أي يترك كما هو) و ننتقل لليمين. هكذا حتى نقابل b
- عند ال b نقوم باستبداله ب a (لأننا نقوم بالإزاحة لذلك مكان ال b فعليا هو ال a السابقة التي تمت إزاحتها) و ننتقل لليمين و ننتقل لحالة جديدة

- ثم نبقى في الحالة الجديدة ما دمنا بـ b حيث نستبدل بـ b وننتقل لليمين لكن نبقى في نفس الحالة
- بعدها عند مصادفة a نستبدل بـ b (لنفس سبب استبدال b بـ a) وننتقل لليمين و ننتقل للحالة السابقة التي تبقى نفسها عند إيجاد a
- عندما نصل لرمز فارغ نقوم باستبداله بالرمز الذي أتينا منه (إذا صادفنا رمز فارغ و كنا في الخطوة السابقة عند a فنستبدله بـ a و الأمر ذاته لل b) و لكن هنا وصولنا للرمز الفارغ يعني الوصول لنهاية السلسلة لذلك نجري الاستبدال و نتحرك لليساار
- بعدها نستمر بالتحرك لليساار دون تغيير أي شيء حتى نصل للرمز \$ فننتقل لليمين خطوة وهذا لنعود لبداية السلسلة من جديد و نستكمل بقية ال TM (نصف الدائرة في النهاية تدل على وجود بقية ال TM)
- (حيث أن هذه الإجرائية تتم قبل تنفيذ TM معينة أي أن هذه الإجرائية لا تقوم بحل مشكلة بحد ذاتها و إنما هي خطوة سابقة فقط)



النهاية

ليس الحلم ما تراه في نومك، الحلم هو ما
يمنعك من النوم

